

Gesture-Controlled WebGL Game — Project Plan

Concept: A browser-based WebGL game in the spirit of Google's AI Quest, where all player input comes from hand gestures captured via webcam instead of keyboard, mouse, or touch.

Timeline: 7 days **Team:** 5 members **Deliverable:** A playable, hosted web build that runs in a desktop browser with a webcam.

1. Team & Ownership

Each system has exactly one owner. Overlapping ownership is the fastest way to lose a day to merge conflicts, so the two WebGL roles are split by system rather than shared.

Member	Owns	Deliverable
Jibran	Game design, asset sourcing, content	Design doc, gesture-to-action mapping, sourced reference/audio assets, level layout
Asif	Rendering & engine	Scene setup, camera, lighting, asset loading, performance budget, build pipeline
Attar	Gameplay & UI	Game state machine, interactions, scoring, HUD, menus, calibration screen
Basim	Hand gesture detection	Webcam capture, landmark inference, gesture classification, smoothing, public input API
Arsalan	3D art & asset pipeline	Blender models, web-optimized <code>.glb</code> exports, texture atlases, naming/export standards

Shared responsibility: everyone writes against the agreed interfaces, not against each other's internals.

2. Technical Stack

Layer	Choice	Notes
Rendering	Three.js (WebGL2)	Faster to ship than raw WebGL; drop to raw GL only if a specific effect demands it
Hand tracking	MediaPipe Tasks Vision — <code>HandLandmarker</code>	Runs in-browser, GPU delegate, 21 landmarks per hand, ~30fps
Asset format	glTF 2.0 (<code>.glb</code>)	Draco or Meshopt compressed
Build	Vite	Fast dev server, simple static output
Hosting	Static host (Netlify / Vercel / GitHub Pages)	Must be HTTPS — <code>getUserMedia</code> requires a secure context
Version control	Git, feature branches, one PR per system	No direct commits to <code>main</code>

3. The Input Contract (lock this on Day 1)

This is the most important agreement in the project. The gesture module is the highest-risk component, so the rest of the game must never be blocked on it.

Basim's module exposes exactly this:

```
js

// gestureInput.js
gestureInput.init({ videoElement })           // returns Promise
gestureInput.on('gesture', handler)           // discrete events
gestureInput.getState()                       // continuous poll

// Event payload
{
  type: 'open' | 'fist' | 'pinch' | 'point',
  hand: 'left' | 'right',
  confidence: 0.0 - 1.0,
  position: { x, y },                        // normalised 0-1, screen space
  timestamp: Number
}
```

Rules:

1. A **keyboard fallback driver** implementing the identical interface exists from Day 1. The game is fully playable via keyboard at all times. Gesture input is a swappable driver, not a dependency.
2. Gesture vocabulary is capped at **four gestures**: open palm, fist, pinch, point. No additions without dropping one.
3. Classification is **debounced** — a gesture must hold for ~200ms above a confidence threshold before it fires. Raw per-frame classification feels broken to players.
4. The module emits `lost` and `regained` events when hands leave frame, so the game can pause gracefully.

4. 3D Asset Pipeline (Arsalan)

The webcam inference loop already consumes significant CPU and GPU each frame. The art budget must leave headroom for it.

Export settings

- Format: **glTF 2.0 Binary** (`.glb`) — not FBX, not OBJ
- Apply all transforms before export (scale 1, rotation 0)
- Metric scale, Y-up
- One file per logical asset

Budgets

Item	Limit
Total scene triangles	~60,000
Individual prop	500 - 3,000 tris
Hero/character model	8,000 - 15,000 tris
Prop textures	512 × 512
Environment / hero textures	1,024 × 1,024 max
Draw calls	Under 100

Rules

- **Bake lighting and ambient occlusion into textures.** No realtime shadows unless the frame budget clearly allows it after gesture detection is integrated.
- **Atlas textures aggressively** so the renderer can batch draw calls.
- Use a single shared material where possible; every unique material is a draw call.

- Compress before handing off:

```
bash
```

```
npx @gltf-transform/cli optimize input.glb output.glb --compress draco --text
```

Typically reduces file size 70–90%.

- If total download becomes a problem, convert textures to **KTX2** / **Basis** for GPU-native compression.

Naming convention

```
env_<name>_v<n>.glb      // environment pieces
prop_<name>_v<n>.glb     // interactable props
char_<name>_v<n>.glb     // characters
```

All finals live in `/public/assets/models/`. Working `.blend` files stay in the shared drive, never in the repo.

5. Schedule

Day	Goal	Exit criteria
1	Scope lock & scaffolding	Design doc signed off; gesture API frozen; repo + Vite build running; art style and palette decided
2	Playable grey-box	Full game loop playable with keyboard input on primitive shapes; MediaPipe streaming landmarks to console
3	First gestures + first art	Two gestures classified reliably; first optimized <code>.glb</code> loading in scene
4	Feature complete	All four gestures working and tuned; art assets flowing into the scene; HUD in place
5	Integration day	Gesture driver swapped in as primary input; calibration/onboarding screen; everything merged to <code>main</code>
6	Polish & performance	Frame budget met, audio in, edge cases handled (no camera, no hands, bad lighting)
7	Buffer & ship	Production build deployed, tested on 2+ machines, demo rehearsed

Day 5 is non-negotiable. If integration slips past Day 5, cut features rather than move the date.

6. Definition of Done

- ☐ Runs in Chrome and Edge on a standard laptop webcam
 - ☐ Holds 30fps minimum with gesture detection active
 - ☐ Onboarding screen teaches the gestures before play starts
 - ☐ Handles camera permission denial with a clear message and keyboard fallback
 - ☐ Recovers gracefully when hands leave frame
 - ☐ Total initial download under 10 MB
 - ☐ Deployed to a public HTTPS URL
-

7. Risks & Mitigations

Risk	Impact	Mitigation
Gesture detection unreliable under poor lighting	High	Confidence thresholds, onboarding lighting hint, keyboard fallback always present
Frame rate collapse from inference + rendering	High	Strict art budget; run inference on a throttled interval (e.g. 20Hz) decoupled from render loop
Scope creep in game design	High	Design frozen Day 1; any addition requires a removal
Integration left to the end	Critical	Keyboard-driver-first architecture; forced integration on Day 5
Free 3D movement via gestures feels bad	Medium	Prefer gesture-as-action (grab, select, cast) on a fixed or auto-moving camera
Asset handoff bottleneck	Medium	Naming convention + shared folder agreed Day 1; placeholder primitives used until finals land

8. Open Decisions

These must be resolved by end of Day 1:

- Game loop type** — on-rails experience, puzzle sequence, or free movement? This determines the entire art and gameplay workload.
- Gesture-to-action mapping** — which of the four gestures does what.
- Camera model** — fixed, orbiting, or auto-moving.
- Session length** — a single 2-3 minute run, or multiple levels.

5. **Art direction** — stylised low-poly is strongly recommended over realism at this timeline.